ClimateLens

คู่มือทดลองใช้งานและติดตั้งระบบ

Group 1 — Anawat Wongprachanukul 66070217 · Amorntep Thongplum 66070219 · Pubeth Sriwattana 66070158 · Teerapat Tuanpromrat 66070093

ClimateLens รับความคิดเห็นปลายเปิดภาษาไทยจากแบบสำรวจพนักงาน (CSV) แล้ว (1) จำแนก**ความรู้สึก** (Positive / Neutral / Negative) (2) จัด**หัวข้อ** 7 หมวดด้วย LLM ที่รันบนเครื่องเอง (3) คำนวณ **Climate Index** แยกตามหัวข้อ แผนก และเดือน แล้วแสดงบน Dashboard

เลือกวิธีทดลองได้ 3 แบบ เรียงจากเร็วที่สุด:

วิธี	ต้องมี	เวลา
ส่วนที่ 2: ใช้เว็บที่ deploy แล้ว	เบราว์เซอร์ + เครือข่ายคณะ IT	2 นาที
ส่วนที่ 3: รันบนเครื่องตัวเอง (ใช้ฐานข้อมูลของกลุ่ม)	Python 3.12, Ollama	20–30 นาที
ส่วนที่ 4: ติดตั้งทั้งระบบ (Supabase ของผู้ทดสอบเอง)	+ บัญชี Supabase	45–60 นาที

ส่วนที่ 1: ภาพรวมระบบและผลของโมเดล

1. **Upload CSV** — คอลัมน์ `feedback` (บังคับ), `department`, `date` (`YYYY-MM-DD`)
2. **Sentiment** — Character TF-IDF (2–5 ตัวอักษร) + Logistic Regression เทรนบน Wisesight Sentiment Corpus (`models/sentiment.joblib`)
3. **Topic** — Typhoon 2.5 (Qwen3 4B) แบบ zero-shot รันผ่าน Ollama ความคิดเห็นไม่ถูกส่งไป LLM บนคลาวด์
4. **Save** — เก็บลง Supabase (PostgreSQL) แยกข้อมูลรายผู้ใช้ด้วย Row Level Security
5. **Dashboard** — Climate Index $= 50 + 50\,(P - N)/T$; ≥ 60 Healthy, 45–60 Watch, < 45 Needs attention; กลุ่มที่มีน้อยกว่า 10 ความเห็นไม่แสดงสถานะ

ตารางที่ 1 ผลการประเมินโมเดล (ตัวเลขเดียวกับสไลด์นำเสนอ)

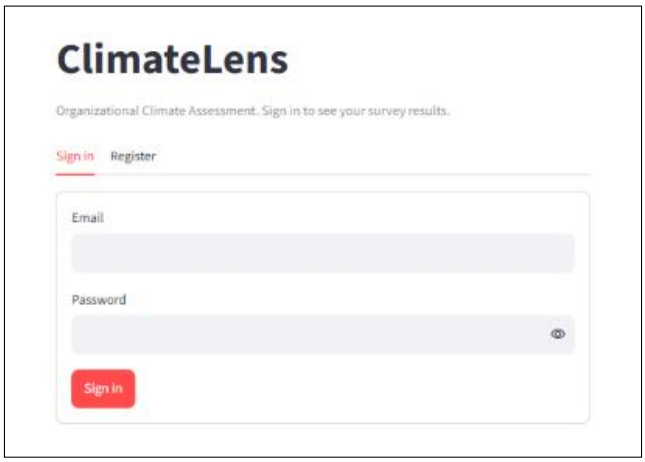
งาน	ชุดทดสอบ	Accuracy	Macro F1
Sentiment (โมเดลที่ใช้)	Wisesight test 2,614 แถว	69.7%	0.668
Sentiment baseline (ตอบ Neutral ทุกแถว)	Wisesight test 2,614 แถว	55.6%	0.238
Topic — Typhoon 4B zero-shot (ที่ใช้ในแอป)	ข้อสอบ 105 ข้อ (7 หมวด × 15)	86.7%	0.867
Topic — Typhoon 4B few-shot	ข้อสอบ 105 ข้อ	87.6%	0.886

> **ข้อจำกัดที่ทราบ:** โมเดล Sentiment เทรนจากโพสต์โซเชียลมีเดีย แต่ใช้กับข้อความในที่ทำงาน (Positive F1 = 0.52) จึงมักอ่านความเห็นเชิงบวกเป็น Neutral — จะเห็นได้ในไฟล์ทดลองด้านล่าง

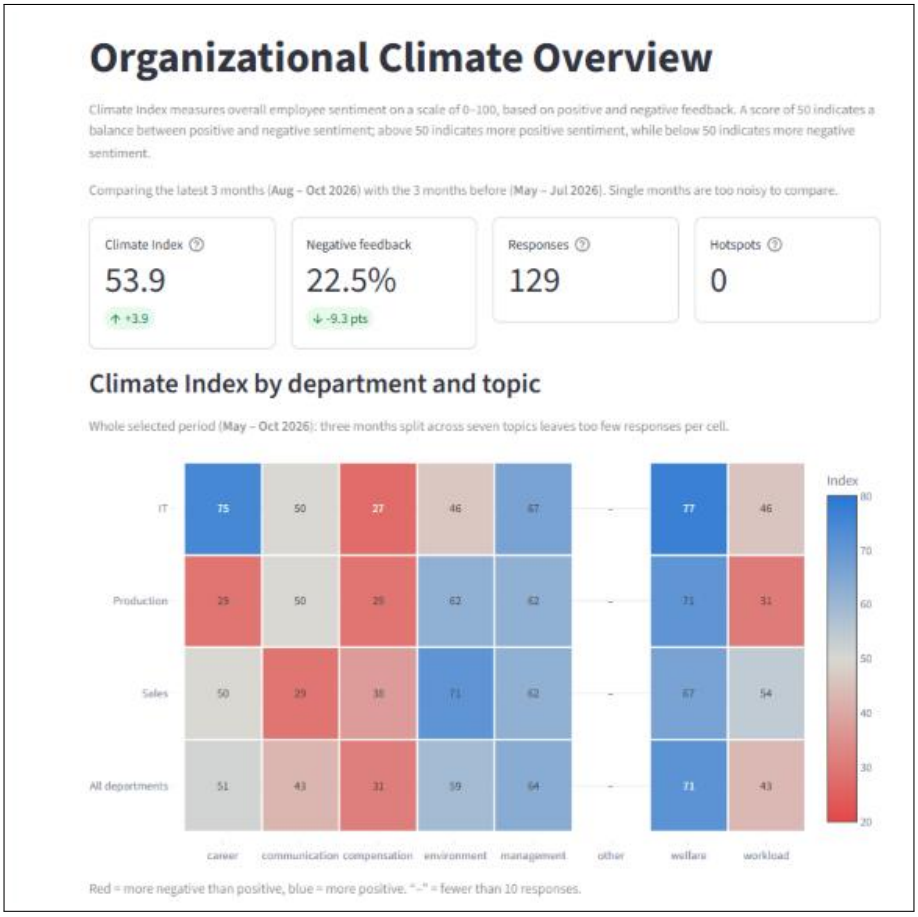
ส่วนที่ 2: ทดลองบนเว็บที่ deploy แล้ว

> เว็บอยู่บน VM ของคณะ เข้าได้จาก**เครือข่ายคณะ IT / VPN ของ KMITL เท่านั้น**

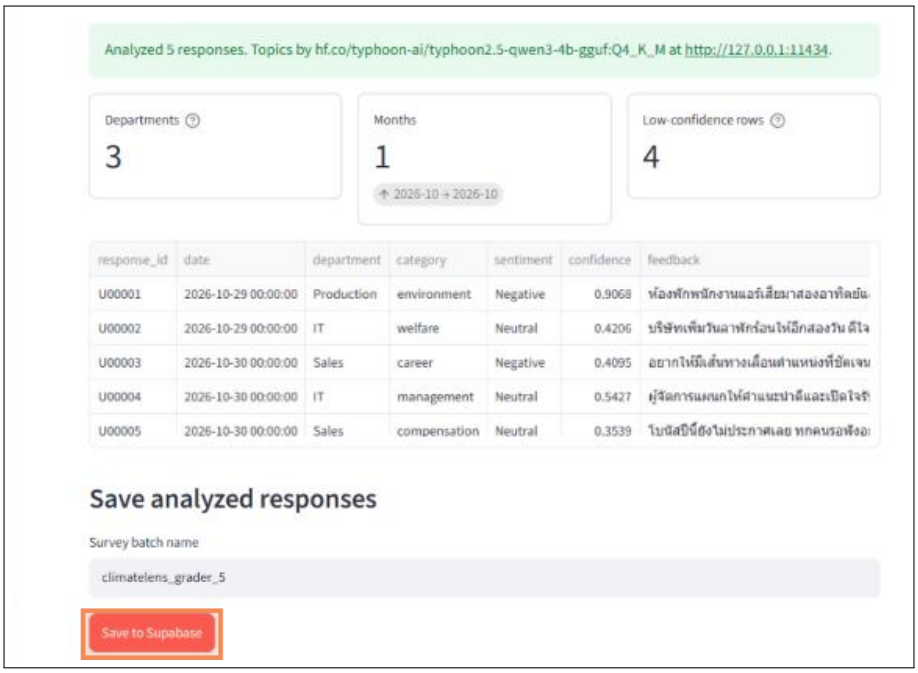
1. เปิด `http://ito.it.kmitl.ac.th:8501` (หรือ `http://10.70.38.15:8501`)
2. เข้าสู่ระบบด้วยบัญชีทดลอง
    a. Email: (ดูในข้อความที่ส่งใน Google Classroom)
    b. Password: (ดูในข้อความที่ส่งใน Google Classroom)
3. บัญชีนี้มีข้อมูลตัวอย่าง 258 ความเห็น (พ.ค.–ต.ค. 2026, 3 แผนก) ดูได้ทันทีที่หน้า **Overview**, **Departments**, **Trends**, **Feedback Explorer**
4. ทดลองอัปโหลด: ไปที่ **Upload & Analyze** แล้วเลือกไฟล์จากโฟลเดอร์ `demo/try/`
    a. `climatelens_grader_5.csv` — 5 แถว ประมาณ 20 วินาที
    b. `climatelens_demo_live_10.csv` — 10 แถว ประมาณ 40 วินาที
5. ตรวจผลในตาราง แล้วกด **Save to Supabase** (ไม่บังคับ) จากนั้นกด **Use this data in the dashboard** เพื่อดูไฟล์ที่เพิ่งอัปโหลดบน Dashboard



รูปที่ 1 หน้าเข้าสู่ระบบ



รูปที่ 2 หน้า Overview: Climate Index แยกแผนก × หัวข้อ



รูปที่ 3 หน้า Upload & Analyze หลังวิเคราะห์ไฟล์ 5 แถว

> **เรื่องที่ควรสังเกต:** Climate Index รวมของ 3 เดือนล่าสุดคือ 53.9 (ดูปกติ) แต่ช่อง Production × Workload ได้ 31 (สีแดง) — นี่คือเหตุผลที่ต้องแยก Climate Index ตามหัวข้อ

ส่วนที่ 3: รันบนเครื่องตัวเอง (ใช้ฐานข้อมูลของกลุ่ม)

ใช้ Supabase ของกลุ่ม จึงไม่ต้องตั้งค่าฐานข้อมูล เข้าสู่ระบบด้วยบัญชีทดลองเดียวกับส่วนที่ 2

3.1 ติดตั้งโปรแกรม

1. ติดตั้ง Python 3.12 (`https://www.python.org/downloads/`)
2. ติดตั้ง Ollama (`https://ollama.com/download`) แล้วเปิดโปรแกรมทิ้งไว้
3. ดาวน์โหลดโมเดล Typhoon (ประมาณ 2.5 GB, ใช้ RAM ราว 4 GB):

```
ollama pull hf.co/typhoon-ai/typhoon2.5-qwen3-4b-gguf:Q4_K_M
```

4. แตกไฟล์ `group_01.zip` หรือ clone repository:

```
git clone https://github.com/AAnawat/Applied-AI-Project.git
cd Applied-AI-Project
```

5. สร้าง virtual environment และติดตั้ง package (รันที่โฟลเดอร์หลักของโปรเจกต์)

```
python -m venv .venv
.venv\Scripts\activate        (Windows)
source .venv/bin/activate      (macOS / Linux)
pip install -r requirements.txt
```

3.2 ตั้งค่าการเชื่อมต่อ

1. คัดลอก `.streamlit/secrets.toml.example` เป็น `.streamlit/secrets.toml`
2. แก้สองบรรทัดแรกเป็นค่าของกลุ่ม:

```
supabase_url = "https://<project-ref>.supabase.co"
supabase_key = "sb_publishable_..."
```

ค่าจริงของทั้งสองบรรทัดอยู่ในข้อความที่ส่งใน Google Classroom

> `supabase_key` นี้คือ **Publishable key** ซึ่ง Supabase ออกแบบให้เปิดเผยได้ (ฝังอยู่ในหน้าเว็บทั่วไป) ข้อมูลถูกป้องกันด้วย Row Level Security — ผู้ใช้แต่ละคนเห็นเฉพาะข้อมูลของตัวเอง ส่วน **Secret key** ไม่ได้แจกให้ และแอปจะปฏิเสธถ้าใส่ Secret key

3.3 เปิดแอป

1. รันจาก**โฟลเดอร์หลักของโปรเจกต์** (ไม่ใช่ใน `dashboard/`):

```
python -m streamlit run dashboard/app.py
```

2. เปิด `http://localhost:8501` แล้วเข้าสู่ระบบด้วยบัญชีทดลอง
3. ทดลองอัปโหลดตามข้อ 4–5 ของส่วนที่ 2

ส่วนที่ 4: ติดตั้งทั้งระบบด้วย Supabase ของตัวเอง

ใช้เมื่อต้องการฐานข้อมูลแยกของตัวเองทั้งหมด ทำส่วนที่ 3.1 ให้เสร็จก่อน

4.1 สร้างโปรเจกต์ Supabase

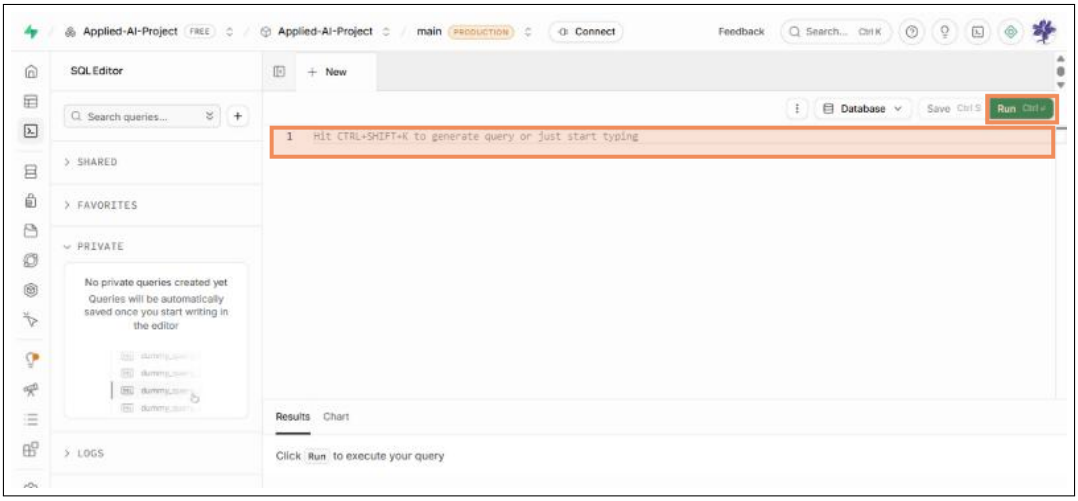
1. สร้างโปรเจกต์ใหม่ตาม Lab 12 Section 1 (**New project**, Region: Singapore, เปิด **Enable Data API**)



รูปที่ 4 ปุ่ม New project ในหน้า Projects

4.2 สร้างตารางด้วย SQL

1. ไปที่เมนู **SQL Editor** แล้วกด **+ New**
2. เปิดไฟล์ต่อไปนี้ทีละไฟล์ **ตามลำดับ** คัดลอกทั้งไฟล์ไปวาง แล้วกด **Run**
   a. `supabase/schema.sql` — ตาราง, view และสิทธิ์
   b. `supabase/migrations/20261025000000_auth_rls.sql` — เจ้าของข้อมูล + Row Level Security
   c. `supabase/migrations/20261025120000_survey_date.sql` — วันที่ของแบบสำรวจ
3. แต่ละไฟล์ต้องขึ้น `Success. No rows returned`

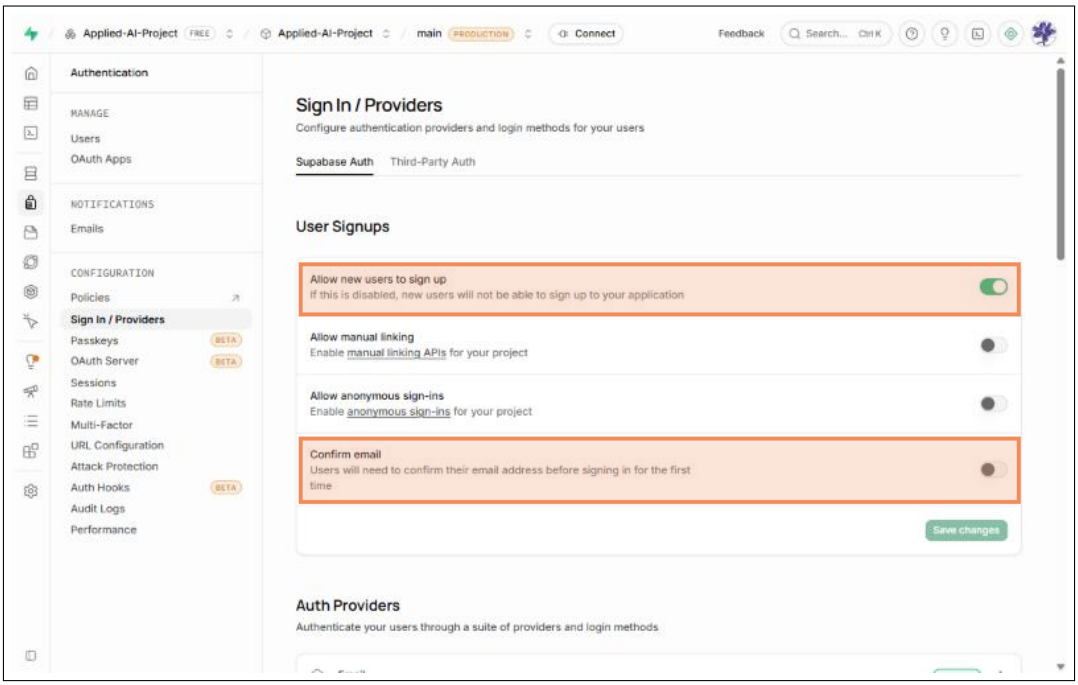


รูปที่ 5 SQL Editor: วาง SQL แล้วกด Run

> ไฟล์ที่ 2 มีตัวแปร `owner_email` ไว้ย้ายข้อมูลเก่า — โปรเจกต์ใหม่ยังไม่มีข้อมูล **ไม่ต้องแก้**

4.3 ตั้งค่าการเข้าสู่ระบบ

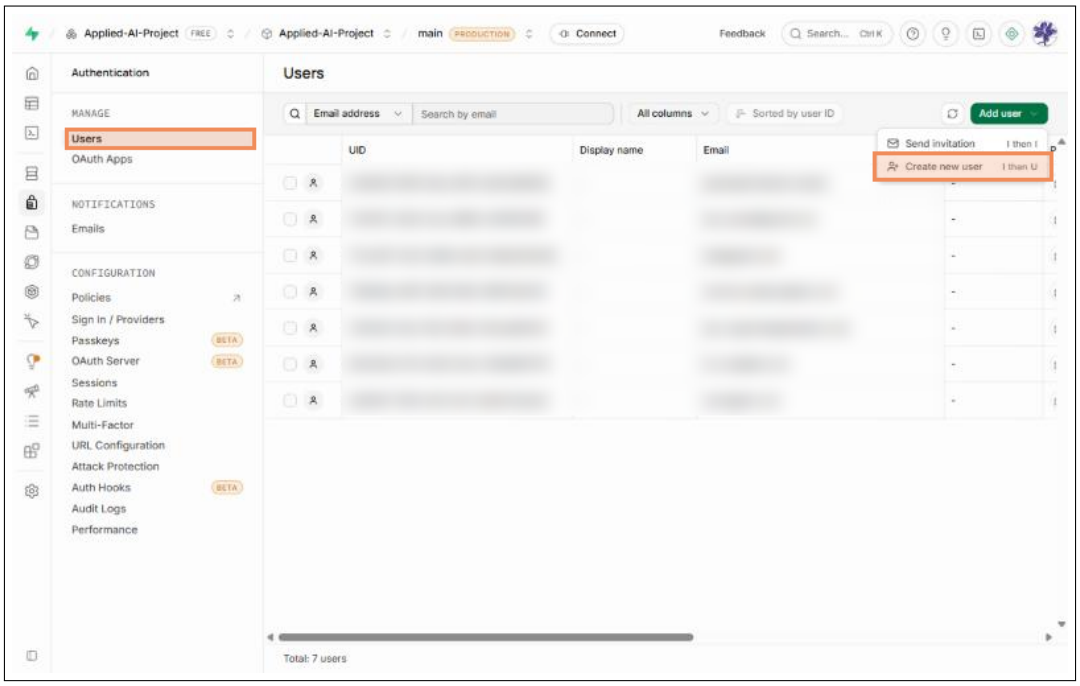
1. ไปที่ **Authentication → Sign In / Providers**
2. เปิด **Allow new users to sign up** ถ้าต้องการให้สมัครเองได้จากแท็บ **Register** ในแอป
3. ปิด **Confirm email** (ระบบส่งอีเมลของ Supabase ส่งได้ไม่กี่ฉบับต่อชั่วโมง) แล้วกด **Save changes**



รูปที่ 6 Authentication → Sign In / Providers

4.4 สร้างผู้ใช้

1. ไปที่ **Authentication** → **Users** → **Add user** → **Create new user**
2. กรอก Email และ Password, ติ๊ก **Auto confirm user?** แล้วกด **Create user**



รูปที่ 7 เมนู Add user → Create new user

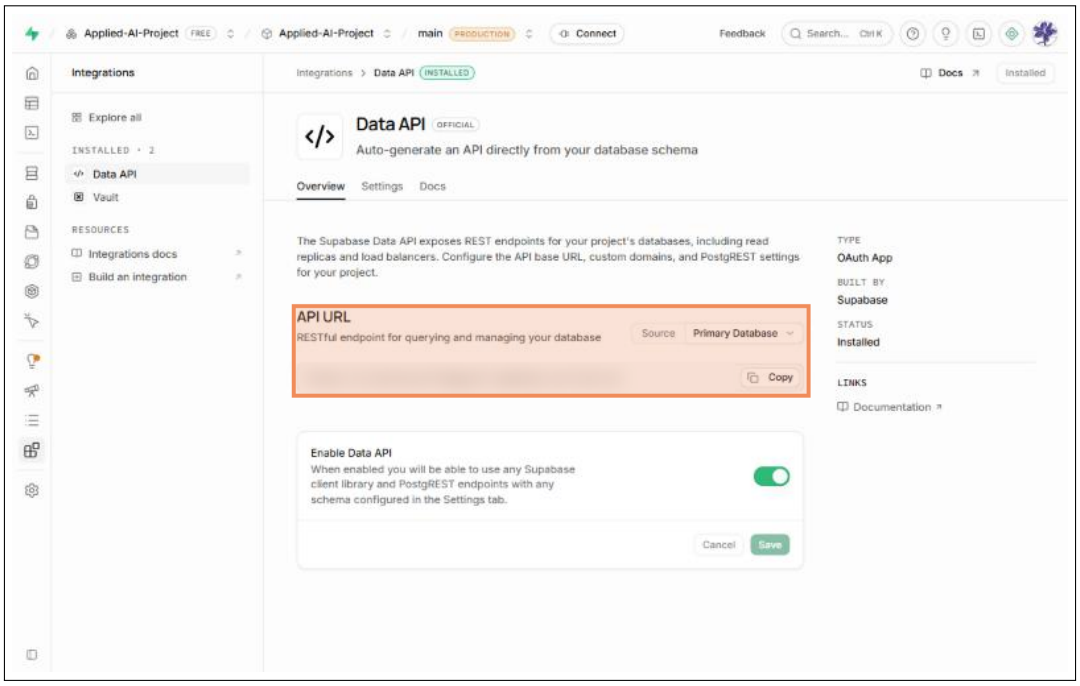


รูปที่ 8 ฟอร์มสร้างผู้ใช้ (ติ๊ก Auto confirm user)

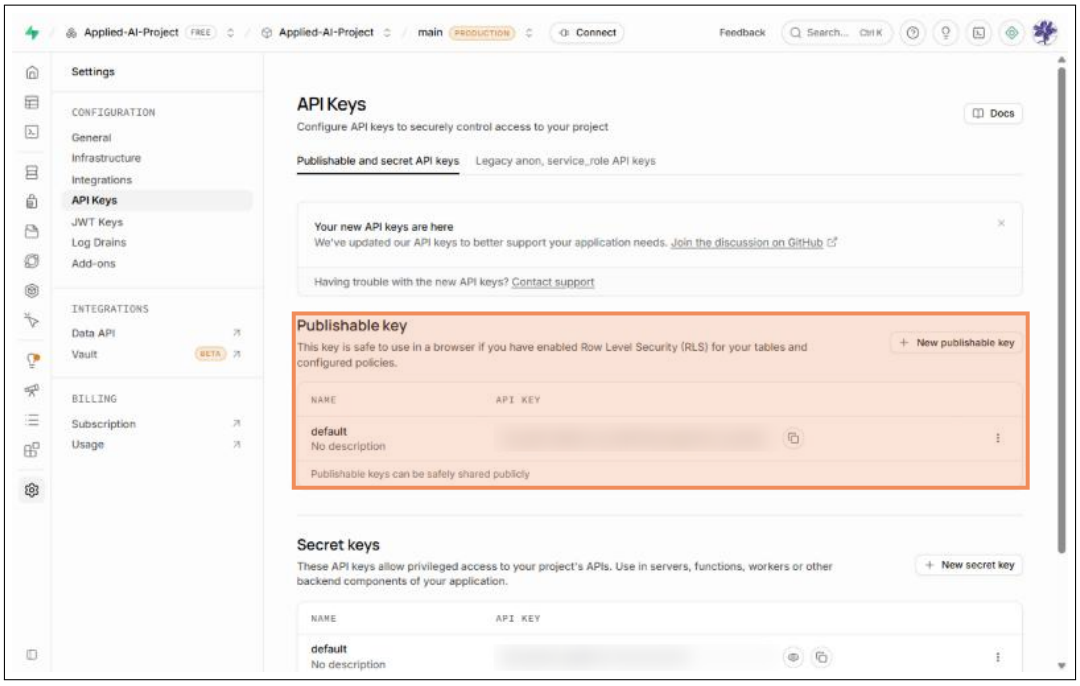
4.5 คัดลอก URL และ Publishable key

1. ไปที่ **Project Settings** → **Data API** คัดลอก **API URL** แล้ว**ตัด** `/rest/v1/` **ท้ายออก** ให้เหลือ `https://<project-ref>.supabase.co`

2. ไปที่ **Project Settings** → **API Keys** คัดลอก **Publishable key** (`sb_publishable_...`) — **ห้ามใช้ Secret key**
3. ใส่ทั้งสองค่าใน `.streamlit/secrets.toml` ตามส่วนที่ 3.2 แล้วเปิดแอปตามส่วนที่ 3.3



รูปที่ 9 Data API: API URL



รูปที่ 10 API Keys: ใช้ Publishable key เท่านั้น

4.6 ใส่ข้อมูลตัวอย่าง

1. เข้าสู่ระบบในแอปด้วยผู้ใช้ที่สร้างในข้อ 4.4
2. หน้า **Upload & Analyze**: อัปโหลดไฟล์ใน `demo/history/` ทีละเดือน (พ.ค. → ต.ค.) แล้วกด **Save to Supabase** ทุกไฟล์
3. ใช้**ไฟล์รายเดือน 6 ไฟล์ หรือ ไฟล์รวม 258 แถว อย่างใดอย่างหนึ่ง** — ถ้าใช้ทั้งคู่ข้อมูลจะซ้ำ

> เวลาวิเคราะห์หัวข้อด้วย CPU: ประมาณ 1.3–3.6 วินาที/แถว (ไฟล์รายเดือน 43 แถว ≈ 1–2.5 นาที) ถ้ารีเฟรชหน้าระหว่างวิเคราะห์ จะต้องเริ่มไฟล์นั้นใหม่

ส่วนที่ 5: Deploy บนเซิร์ฟเวอร์ด้วย Docker

ใช้ `docker-compose.yml` รันแอป + Ollama บนเครื่องเดียว (ทดสอบบน Ubuntu 24.04, 8 vCPU, 8 GB RAM, ไม่มี GPU)

1. คัดลอกโปรเจกต์ไปที่เซิร์ฟเวอร์ และสร้าง `.streamlit/secrets.toml` ตามส่วนที่ 3.2
2. เพิ่มบรรทัดนี้ใน `secrets.toml` ให้แอปคุยกับ container ของ Ollama:

```
ollama_host = "http://ollama:11434"
```

3. รันที่โฟลเดอร์หลักของโปรเจกต์ (ครั้งแรกจะดาวน์โหลดโมเดล 2.5 GB):

```
docker compose up -d --build
docker compose ps
```

4. `docker compose ps` ต้องแสดง `app` เป็น `healthy`
5. เปิด `http://<server-ip>:8501`

ส่วนที่ 6: ปัญหาที่พบบ่อย

ข้อความที่เห็น	วิธีแก้
`Set supabase_url and supabase_key in .streamlit/secrets.toml`	ยังไม่ได้สร้าง `secrets.toml` หรือรันจากโฟลเดอร์ `dashboard/` — รันจากโฟลเดอร์หลัก
`The configured Supabase key is a Secret/service_role key`	ใส่ Secret key ผิดช่อง — เปลี่ยนเป็น Publishable key
`Wrong email or password.`	ตรวจบัญชี หรือยังไม่ได้ติ๊ก Auto confirm ตอนสร้างผู้ใช้
`Could not analyze topics with Local LLM`	Ollama ไม่ได้เปิด หรือยังไม่ได้ `ollama pull` โมเดล
`Missing `feedback` column`	CSV ต้องมีหัวคอลัมน์ `feedback` และบันทึกเป็น UTF-8
`Please remove result columns before analyzing`	ลบคอลัมน์ `sentiment` / `category` ออกจาก CSV
บันทึกไม่ได้ (code `42501`)	ยังไม่ได้รัน SQL ไฟล์ที่ 2 (`auth_rls`)
บันทึกไม่ได้ (code `PGRST204`)	ยังไม่ได้รัน SQL ไฟล์ที่ 3 (`survey_date`)